

Урок 19. Корутины. Flow. Введение

21 августа 2020

В этом уроке разбираемся что такое Flow, как он работает под капотом и чем отличается от каналов.

Что такое Flow

Flow - это упакованный в объект фрагмент кода, который производит какой-то последовательный набор элементов (данных). С одной стороны есть условный Отправитель, который этот код создал и готов предоставить его в виде Flow обертки. А с другой стороны есть Получатель, который возьмет этот Flow, запустит его и предоставит ему специальный колбэк, куда Flow будет складывать данные, который он производит.

Если вы знаете RxJava, то Flow - это аналог Observable/Flowable. Т.е. это некая фабрика, которая умеет производить данные. При создании она ничего не делает. Но как только мы ее запустим, она начнет работу и будет постить результаты в колбэк, который мы ей предоставим.

В качестве примера можно рассмотреть библиотеку по работе с файловой системой. Предположим, что у нее есть функция поиска файла. Мы даем ей имя файла, она начинает шерстить файловую систему и выдавать нам подходящие результаты по мере поиска. Этот алгоритм может быть упакован в Flow объект и передан нам. Т.е. при вызове функции поиска, библиотека не начинает работу и не дает нам результаты поиска. Она дает нам Flow обертку, которая содержит код поиска. Когда мы решим выполнить поиск, мы запустим этот Flow и он начнет искать. И по мере выполнения работы он будет отправлять результаты в колбэк, который мы ему предоставили.

В общем то, я сейчас одно и тоже написал три раза, используя немного разные формулировки. Но чтобы уж наверняка понять, что же такое Flow, давайте рассмотрим практический пример, как от обычной функции прийти к Flow. Только пойдем мы не сразу, а по шагам. Чтобы было понятнее, зачем все это нужно.

От обычной функции к Flow

В качестве примера возьмем поиск авиарейсов. Предположим, что мы написали некую функцию агрегатор, которая по определенным критериям ищет рейсы.

Код функции:

```
1 fun searchFlight(criteria: Criteria): List<Flight> {
2
3     val results = mutableListOf<Flight>()
4
5     for (airCarrier in getAirCarriers()) {
6         val flight = airCarrier.searchFlight(criteria)
7         results.add(flight)
8     }
9
10    return results
11 }
```

Формируется список авиаперевозчиков, и для каждого из них выполняется поиск. Найденный рейс добавляется в коллекцию, которая затем возвращается, как результат работы функции. Обработку ошибок и ненайденных рейсов опустим, чтобы не усложнять код.

Использование этой функции на стороне клиента выглядит так:

```
1 val flights = searchFlight(criteria)
2 showFlights(flights)
```

Вызываем ее, получаем полный список рейсов через какое-то время и выводим их на экран.

В целом все неплохо, но от клиентов, которые вызывают эту функцию поступило пожелание. Они не хотят долго ждать и получать все рейсы одним списком. Было бы круто выдавать рейсы по одному, по мере их нахождения у авиаперевозчиков. Это позволит начать выводить на экран первые результаты поиска почти сразу же после старта, а не ждать пока поиск полностью завершится.

Не вопрос. Переписываем функцию.

```
1 fun searchFlight(criteria: Criteria, onFind: (Flight) -> Unit) {
2     for (airCarrier in getAirCarriers()) {
3         val flight = airCarrier.searchFlight(criteria)
4         onFind(flight)
5     }
6 }
```

Теперь она на вход кроме критериев поиска принимает колбэк с типом (Flight) -> Unit, в который отправляет рейсы по мере их нахождения.

А клиент использует функцию таким образом:

```
1 searchFlight(criteria) { flight ->
2     showFlight(flight)
3 }
```

Найденные рейсы будут по одному приходить в колбэк и выводиться на экран.

Клиент доволен. Но некоторое время спустя он попросил об еще одном небольшом улучшении. Он хотел бы, чтобы наша функция при запуске не выполняла поиск, а возвращала объект-функцию, которая умеет выполнять поиск. И клиент мог бы у себя в коде ее хранить и передавать, как обычный объект. А когда нужно, он мог бы ее запустить.

Переписываем нашу функцию:

```
1 fun searchFlight(criteria: Criteria): ((Flight) -> Unit) -> Unit {
2     return { onFind ->
3         for (airCarrier in getAirCarriers()) {
4             val flight = airCarrier.searchFlight(criteria)
5             onFind(flight)
6         }
7     }
8 }
```

Мы передаем ей на вход критерии поиска, и она возвращает объект-функцию, которая при ее запуске будет выполнять поиск по этим критериям. Т.е. критерии поиска уже зашиты в ней. Осталось только запустить ее, когда понадобится.

Обратите внимание на тип объекта-функции, создаваемой функцией searchFlight:

```
1 ((Flight) -> Unit) -> Unit
```

Когда клиент будет ее запускать, он на вход передаст колбэк с типом (Flight) -> Unit, чтобы получать результаты поиска (Flight).

Т.е. функция searchFlight создает и возвращает функцию поиска рейсов, на вход которой можно передать функцию (колбэк), куда будут приходить найденные рейсы.

На стороне клиента это выглядит так:

```
1 val search = searchFlight(criteria)
2
3 /// ...
4
5 search.invoke { flight ->
6     showFlight(flight)
7 }
```

Вызывая searchFlight он получает функцию, как обычный объект, и может передать ее куда-либо, если это необходимо. Поиск рейсов при этом не стартует. Когда придет время, клиент запускает полученную функцию (invoke) и передает колбэк для получения результатов. В этот момент и начинается поиск.

Подытожим. Мы поменяли функцию. Вместо обычного результата своей работы, она возвращает нам объект. Этот объект мы можем запустить, как функцию, и получать в колбэк результаты его работы в виде последовательности данных.

И знаете что у нас получилось в итоге? Получился аналог Flow.

Давайте снова перепишем нашу функцию, но в этот раз используем Flow. И вы увидите, что результат будет очень похож. Только возня с колбэком будет спрятана под капотом.

```
1 fun searchFlight(criteria: Criteria): Flow<Flight> {
2     return flow {
3         for (airCarrier in getAirCarriers()) {
4             val flight = airCarrier.searchFlight(criteria)
5             emit(flight)
6         }
7     }
8 }
```

Оператор flow создает объект Flow (объект-функция в нашем примере выше), который умеет выполнять поиск рейса. Результаты он будет отправлять в свой стандартный метод emit. Этот метод будет перенаправлять результаты в колбэк.

Давайте посмотрим, как на стороне клиента запустить Flow и передать ему колбэк.

```
1 val searchFlow = searchFlight(criteria)
2
3 // ...
4
5 scope.launch {
6     searchFlow.collect { flight ->
7         showFlight(flight)
8     }
9 }
```

Методом searchFlight создаем Flow и позже в корутине его запускаем методом collect, передавая туда колбэк для получения данных. Абсолютно та же схема, что и придуманная нами ранее. Отличия только в том, что Flow заточен под корутины. А метод collect и код внутри оператора flow - это suspend функции. И обратите внимание, что создание Flow оператором flow у нас выполняется в обычном коде. А запуск Flow методом collect - только в корутине (или в другой suspend функции).

Чтобы лучше все это понять, давайте посмотрим, как Flow работает изнутри.

Под капотом

Про работу Flow под капотом долго рассказывать не придется, т.к. там просто идет жонглирование функциями, похожее на то, что мы сами делали ранее в примере. Давайте на простом примере посмотрим, как именно это происходит.

Очень простой Flow:

```

1 val flow = flow {
2     // flow block
3     for (i in 1..10) {
4         delay(100)
5         emit(i)
6     }
7 }

```

Он с паузой в 100 мсек отправляет числа от 1 до 10.

Поначалу может показаться, что этот код сразу начинает работать и отправлять куда-то эту последовательность чисел. Но на самом деле это не так. Flow создается и знает, что он должен делать, но пока не делает. Работу он начнет только тогда, когда мы попросим его об этом.

Для этого используется метод collect.

```

1 flow.collect { i ->
2     // collect block
3     log(i)
4 }

```

Именно этот метод и запустит работу. Flow начнет создавать и отправлять элементы в этот блок кода, где мы просто выводим их в лог.

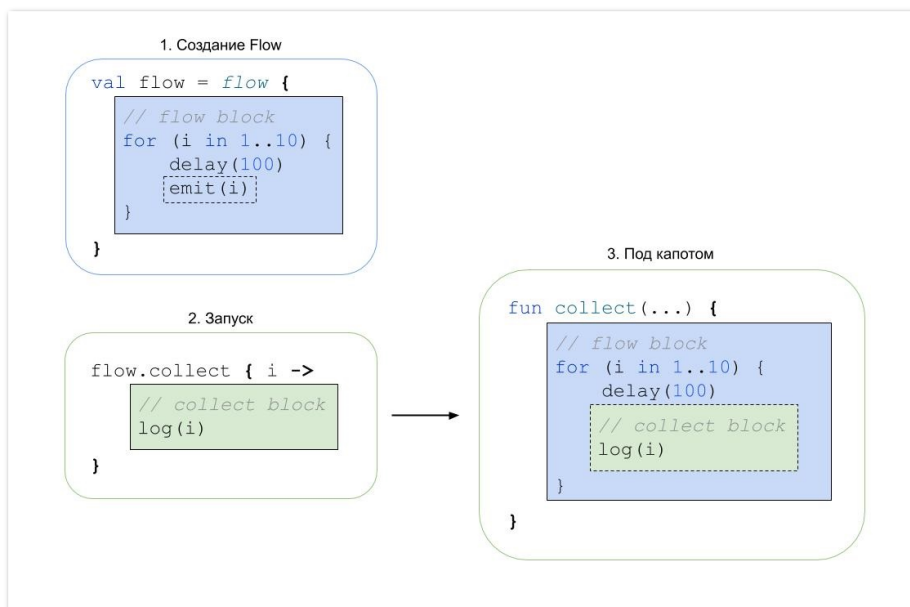
Обратите внимание на комментарии в коде.

Блок кода, который мы передаем в оператор flow, я пометил как блок flow. Этот код генерирует данные.

А блок кода, который мы передаем в метод collect, я пометил, как блок collect. Этот код получает данные.

И ни flow, ни collect не запускают напрямую свои блоки.

Давайте на рисунке посмотрим, что именно происходит.



1. Создается объект Flow, он хранит в себе блок flow, в котором содержится код по генерации данных и отправке их в метод emit.

2. Чтобы запустить работу и начать получать данные Flow, мы вызываем его метод collect и на вход этому методу передаем блок collect, в который мы хотели бы получать данные, создаваемые в блоке flow.

Теперь у Flow есть два блока кода: flow и collect. Их можно считать отправителем и получателем. Осталось стартовать работу отправителя, а результаты передавать получателю.

3. Когда мы запускаем метод collect, Flow берет свой блок flow и запускает его, тем самым стартуя работу. А в методе emit он включает перенаправление в блок collect.

Это приводит к тому, что блок flow начинает генерировать числа и с паузой в 100 мсек отправлять их в emit. А emit в свою очередь шлет их в блок collect.

Корутины

Теперь мы готовы обсудить корутинную составляющую Flow.

У нас есть блоки flow и collect. Один генерирует и отправляет данные, другой их получает и обрабатывает. Каждый из них может работать долго время. И чтобы при запуске в корутинах они не блокировали потоки, эти блоки являются suspend функциями.

Поэтому, когда мы пишем эти блоки, мы должны следовать тем же правилам, что и при написании suspend функций. Мы не должны размещать там код, который блокирует поток.

Теперь обратим внимание, кто именно запускает эти блоки.

1. Метод `collect()` запускает блок `flow`, чтобы начать работу.
2. Блок `flow` запускает блок `collect`, чтобы отправлять данные.

Со вторым пунктом вопросов нет. Блок `flow` без проблем запускает блок `collect`, т.к. они оба `suspend` функции.

А вот с первым пунктом не все так просто. Метод `collect` не может быть обычным методом. Он запускает `suspend` блок кода (`flow`). А значит он сам также должен быть `suspend`. Потому что вызвать `suspend` функцию мы можем только в корутине или в другой `suspend` функции, но не в обычном коде. Поэтому метод `collect` помечен, как `suspend` и может быть вызван только в корутине (или другой `suspend` функции).

Это логично с учетом того, что именно этот метод и выполняет всю работу как по созданию данных, так и по их обработке. Внутри него вызывается блок `flow`, который внутри себя вызывает блок `collect`. В итоге этот метод может работать достаточно долго. И он должен не блокировать поток, а приостанавливать корутину на это время.

Подытожим. Метод `collect` является `suspend`. Внутри себя он запускает и `flow` блок и `collect` блок. Поэтому вся работа по отправке и получению данных будет выполнена в корутине, в которой мы вызываем метод `collect()`.

А метод `flow()` может быть вызван где угодно. Он только создает объект `Flow` и помещает в него блок `flow`. Но он ничего не запускает.

Отличие от каналов

В принципе, подробно узнав, что такое каналы и что такое `Flow`, уже не должно возникать вопроса, чем они отличаются. Это абсолютно разные механизмы. Но давайте все же явно это проговорим.

И с каналом и с `Flow` можно отправлять/получать данные. Возможно, именно поэтому их можно спутать при поверхностном рассмотрении.

Но, канал - это потокобезопасный инструмент для передачи данных между корутинами. Он не создает ничего. Только передает. Т.е. должны существовать отправитель и получатель. Они работают в своих корутинах независимо друг от друга, используя канал для обмена данными.

А `Flow` - это генератор данных. В этом случае нет явного отправителя. Вместо него есть создатель. Он создает `Flow` и дает его получателю. Получатель запускает `Flow` в своей корутине и, можно сказать, сам же становится отправителем.

Мы в этом уроке рассмотрели очень простой пример `Flow`. Дальше в уроках будут встречаться более сложные варианты. И в некоторых из них будут использоваться каналы. Думаю, это поможет еще лучше понимать различие между ними.

Расширенная `suspend` функция

Есть еще один интересный момент, который я хотел бы обсудить. Ничего нового сейчас не скажу, но акцентирую внимание на том, что может быть неочевидно.

Обычно `suspend` функция возвращает нам одно значение. И пока мы ждем это значение, корутина приостанавливается. `Flow` позволяет расширить это поведение. Он делает так, что мы можем получать последовательность (поток) данных вместо одного значения. И это будет происходить в `suspend` режиме. Т.е. корутина будет приостанавливаться на время ожидания каждого элемента.

В принципе мы уже это видели в этом уроке ранее, но давайте взглянем еще раз на примере с поиском рейсов.

Есть такая `suspend` функция:

```
1 suspend fun searchFlight(criteria: Criteria): List<Flight>
```

Она по заданным критериям возвращает список полетов.

Мы запускаем ее в корутине и ждем, пока она отработает и вернет нам данные.

```
1 launch {
2     val flights = searchFlight(criteria)
3 }
```

Ничего необычного. `suspend` функция возвращает одно значение. И пока мы ждем это значение, корутина приостанавливается.

Переписываем функцию, чтобы она возвращала `Flow`:

```
1 fun searchFlight5(criteria: Criteria): Flow<Flight>
```

Теперь эта функция не обязана быть `suspend`.

И вызываем ее в корутине

```
1 launch {
2     searchFlight(criteria).collect { flight ->
3         showFlight(flight)
4     }
5 }
```

В этом случае `suspend` функцией является метод `collect()`. Но он не возвращает нам одно единственное значение. Он позволяет нам получить последовательность данных. И он приостанавливает корутину на время следующего элемента. А когда последовательность закончится, корутина пойдет дальше.

Т.е. Flow расширяет возможности suspend функций, позволяя нам получать последовательность данных в suspend режиме.

Cold Flow

Как вы уже наверно поняли, Flow является холодным источником данных. Он для каждого получателя будет генерировать данные заново.

Т.е. каждый вызов collect будет приводить к тому, что снова будет запускаться блок flow и генерировать элементы. Потому что collect - это всего лишь вызов блока кода flow.